STUDENT RIDE COORDINATION PLATFORM

Scope of Work & Development Estimate — MVP

---

1. Project Overview

The proposed solution is a cross-platform student ride coordination platform designed to help verified university students coordinate shared rides to and from airports.

The platform will allow students to create travel requests, discover compatible riders, automatically form ride groups based on predefined business rules, communicate through real-time chat, coordinate fare sharing, and rate fellow riders after completing a trip.

The MVP will focus on delivering the core ride coordination experience using a **rule-based matching engine** rather than AI-based optimization.

The architecture will be designed in a modular and scalable manner so that AI-based recommendations, online payments, Uber integration, and other advanced capabilities can be introduced in future phases without requiring major redevelopment of the MVP.

---

2. MVP Objectives

The primary objectives of the MVP are to:

- Allow verified university students to create and manage airport travel requests.
- Match compatible students using predefined business rules.
- Automatically organize students into ride groups.
- Allow riders to communicate through real-time group chat.
- Manage the complete ride lifecycle from creation to completion.
- Calculate and coordinate each rider's estimated fare share.
- Provide rider ratings and reliability scores.
- Provide real-time push notifications.
- Provide administrators with centralized platform management.
- Establish a scalable technical foundation for future features.

---

3. Platforms Included

The MVP will include:

Mobile Application

A single cross-platform application developed using React Native for:

- iOS
- Android

Backend

- Backend APIs
- Business logic
- Database
- Authentication services
- Ride matching engine
- Ride and group management
- Notification services
- Chat functionality
- Event logging

Web Administration Panel

A web-based administration portal for managing and monitoring the platform.

Production Deployment

- Backend deployment
- Database configuration
- Production environment configuration
- Mobile application build preparation
- Apple App Store submission support
- Google Play Store submission support

---

4. User Roles

The MVP will support the following primary roles.

4.1 Student / Rider

Students will be able to:

- Register using their university email.
- Verify their email address.
- Create and manage their profile.
- Create travel requests.
- Discover compatible rides.
- Join ride groups.
- Leave eligible ride groups.
- View ride details.
- Participate in group chat.
- View estimated fare share.
- Confirm payment coordination.
- Receive ride notifications.
- Rate fellow riders.
- View their reliability score.
- View their ride history.

4.2 Administrator

Administrators will be able to:

- Manage students.
- Manage universities.
- Manage airports.
- Manage destinations.
- Monitor rides.
- Monitor ride groups.
- Manage travel events.
- Send platform notifications.
- Configure platform settings.
- Review user and ride information.

---

5. Authentication & Account Management

The MVP will include a secure authentication and account management system.

Features

- Student registration.
- University email verification.
- Secure login.
- Logout.
- Password recovery.

- Password reset.
- Session management.
- Account activation/deactivation.
- Basic authentication validation.
- User profile creation.

University Verification

The system will validate that the registered email belongs to an approved university domain.

Administrators will be able to configure supported university email domains through the admin panel.

---

6. Student Profile Management

Students will have a dedicated profile containing relevant ride coordination information.

Profile Information

- Name.
- Profile image.
- University.
- Email address.
- Phone/contact information where applicable.
- Pickup preferences.
- Luggage information.
- Payment coordination information.
- Ride history.
- Completed rides.
- Reliability score.
- Ratings received.

Students will be able to update their permitted profile information.

---

7. Ride / Trip Creation

Students will be able to create a new airport ride request.

Ride Information

The ride creation workflow will capture:

- Travel direction.
- Airport.
- Travel date.
- Flight information.
- Flight time.
- Pickup location/area.
- Number of passengers.
- Luggage information.
- Additional ride notes.
- Ride status.

Travel Direction

The system will support the configured travel directions, such as:

- University → Airport
- Airport → University

Additional directions can be configured as required.

Ride Lifecycle

The system will manage ride states such as:

- Draft
- Open
- Matched
- Grouped
- Confirmed
- In Progress
- Completed
- Cancelled

The final status workflow will be implemented based on the agreed business rules.

---

8. Ride Discovery

Students will be able to discover compatible ride opportunities.

The application will display relevant rides based on:

- University.
- Travel direction.
- Airport.
- Travel date.

- Flight time.
- Pickup area.
- Available vehicle capacity.
- Luggage requirements.
- Ride/group status.

Students will be able to view available ride groups and their relevant details before joining.

---

9. Rule-Based Ride Matching Engine

The MVP will use a deterministic **rule-based matching engine**.

No AI optimization or AI recommendation layer is included in the 270-hour MVP.

Primary Matching Rules

The matching engine will evaluate riders based on:

1. Same university.
2. Same travel direction.
3. Same airport.
4. Same travel date.
5. Flight time within the configured matching window.
6. Available vehicle capacity.
7. Luggage/capacity compatibility.
8. Ride/group availability.
9. Current group status.

Matching Workflow

The system will:

1. Receive a new ride request.
2. Validate the request.
3. Search for compatible existing rides/groups.
4. Apply predefined matching rules.
5. Identify eligible groups.
6. Recommend or assign the compatible group based on the configured business logic.
7. Create a new group when no suitable group exists.
8. Update the ride/group status.
9. Notify affected users.
10. Log the matching event.

Group Capacity

The system will enforce configured rider and vehicle capacity limits.

A group will not be allowed to exceed the configured capacity.

Join / Leave Handling

The system will support:

- Joining an eligible ride group.
- Leaving an eligible group.
- Group capacity validation.
- Updating available capacity.
- Re-evaluating group status where required.
- Notifications when membership changes.

---

10. Ride Group Management

Once compatible riders are identified, they will be organized into ride groups.

Group Features

- Group creation.
- Group member management.
- Rider limit management.
- Group status.
- Ride information.
- Pickup information.
- Flight information.
- Fare information.
- Booker information.
- Group chat.
- Group notifications.

Booker Workflow

The MVP will support a designated **booker/responsible rider** workflow.

The exact business process may include:

- Identifying the rider responsible for coordinating the booking.
- Displaying the booker's details to group members.
- Coordinating the estimated fare.
- Recording payment confirmation.
- Managing booking-related ride information.

The MVP will coordinate payment confirmation but will not process online payments directly.

---

11. Ride Lifecycle Management

The platform will maintain a complete ride lifecycle.

The system will support appropriate transitions between:

- Created
- Open
- Matched
- Grouped
- Confirmed
- In Progress
- Completed
- Cancelled

Business rules will determine which status transitions are permitted.

The backend will record relevant lifecycle events for auditing and future analytics.

---

12. Fare Calculation & Fare Split

The MVP will provide fare coordination functionality.

Included

- Estimated total ride cost.
- Number of riders.
- Individual rider share.
- Fare split calculation.
- Booker/payment responsibility tracking.
- Payment confirmation status.
- Ride-level fare information.

Important Limitation

The MVP will **not process actual online payments**.

There will be no:

- Stripe payment gateway.

- Automated card payments.
- Automated refunds.
- Payment processor settlement.
- Platform payment fees.

The MVP will provide a coordination and confirmation workflow only.

---

13. Real-Time Group Chat

Each active ride group will have a dedicated real-time chat.

Features

- Group chat.
- Send text messages.
- Receive messages in real time.
- Message timestamps.
- Group member visibility.
- Ride information reference.
- Basic message history.
- Chat notifications.
- System-generated ride updates where applicable.

Pinned Ride Information

Important ride information will remain accessible within the group conversation, including:

- Airport.
- Date.
- Flight time.
- Pickup location.
- Group members.
- Fare information.
- Booker information.

---

14. Push Notifications

The platform will provide push notifications for important ride and account events.

Notification Types

Notifications may include:

- New ride match.
- Ride group created.
- User joined a group.
- User left a group.
- New group member.
- Chat message.
- Ride status update.
- Booking reminder.
- Ride reminder.
- Fare/payment confirmation.
- Rating reminder.
- Administrative announcements.

Users will receive notifications through the mobile application.

---

15. Ratings & Reliability System

After completion of a ride, users will be able to rate other participating riders.

Features

- Post-ride rating.
- Rider reliability rating.
- Rating history.
- Reliability score calculation.
- Display reliability score on profile.
- Prevention of duplicate ratings for the same ride where applicable.

The reliability system will be based on completed ride participation and submitted ratings.

---

16. Ride History

Students will have access to their previous ride activity.

Ride History Includes

- Upcoming rides.
- Active rides.
- Completed rides.

- Cancelled rides.
- Ride date.
- Airport.
- Travel direction.
- Group information.
- Fare information.
- Ride status.

---

17. Travel Event Calendar

The platform will include a travel event calendar to help students identify high-demand travel periods.

Calendar Features

- University holidays.
- Travel periods.
- Important university dates.
- Peak travel dates.
- Event title.
- Event description.
- Event date.
- Event visibility.

Administrators will manage calendar events through the admin panel.

The calendar will help students plan rides in advance.

---

18. Admin Panel

A secure web-based administration panel will be provided for platform management.

18.1 Dashboard

The dashboard will provide an overview of platform activity.

Potential information includes:

- Total students.
- Active students.
- Total rides.

- Active rides.
- Completed rides.
- Cancelled rides.
- Active groups.
- Upcoming travel events.

18.2 User Management

Administrators will be able to:

- View students.
- Search students.
- Filter students.
- View student profiles.
- View ride history.
- View reliability information.
- Activate/deactivate accounts.
- Manage basic account information.

18.3 Ride Management

Administrators will be able to:

- View rides.
- Search rides.
- Filter rides.
- View ride details.
- View ride status.
- View participating riders.
- View group information.
- Monitor cancelled/completed rides.

18.4 Ride Group Monitoring

Administrators will be able to:

- View active groups.
- View group members.
- View group capacity.
- View ride details.
- Monitor group status.

- Review relevant ride information.

18.5 University Management

Administrators will be able to configure:

- University name.
- University email domain.
- University status.
- Basic university information.

This will allow the platform to support additional universities in future phases.

18.6 Airport Management

Administrators will be able to:

- Add airports.
- Edit airports.
- Activate/deactivate airports.
- Configure airport information.

18.7 Destination Management

Administrators will be able to:

- Add destinations.
- Edit destinations.
- Activate/deactivate destinations.
- Manage destination information.

18.8 Travel Event Management

Administrators will be able to:

- Create events.
- Edit events.
- Delete/deactivate events.
- Configure event dates.

- Add event descriptions.
- Manage university travel periods.

18.9 Push Notification Management

Administrators will be able to send platform-wide announcements.

Examples:

- Travel reminders.
- System announcements.
- University announcements.
- Important service updates.

18.10 Platform Settings

Administrators will be able to manage configurable platform settings, including applicable:

- Matching time windows.
- Group capacity.
- Ride configuration.
- Notification settings.
- Basic platform configuration.

---

19. Backend & API Development

The backend will provide the APIs and business logic required by the mobile application and administration panel.

Backend Components

- Authentication APIs.
- User APIs.
- Profile APIs.
- University APIs.
- Airport APIs.
- Destination APIs.
- Ride APIs.
- Matching APIs.
- Group APIs.

- Chat APIs.
- Notification APIs.
- Rating APIs.
- Fare calculation APIs.
- Calendar APIs.
- Admin APIs.

Backend Business Logic

The backend will handle:

- Authentication.
- Authorization.
- Data validation.
- Matching rules.
- Group capacity.
- Ride lifecycle.
- Fare calculation.
- Notifications.
- Ratings.
- Event logging.
- Administrative permissions.

---

20. Database Design

The database architecture will support the core MVP functionality and future expansion.

The data model will include appropriate entities such as:

- Users.
- Universities.
- Airports.
- Destinations.
- Rides.
- Ride Groups.
- Group Members.
- Messages.
- Ratings.
- Fare Records.
- Notifications.
- Travel Events.
- Platform Settings.
- Ride/Event Logs.

The database will be structured to support future expansion to additional universities, airports, destinations, payment systems, and recommendation services.

---

21. Event Logging & Audit Data

The MVP will include basic event logging for important ride operations.

Events may include:

- Ride created.
- Ride updated.
- Ride matched.
- Group created.
- Rider joined.
- Rider left.
- Ride status changed.
- Ride completed.
- Ride cancelled.
- Fare confirmation.
- Rating submitted.

This data will provide a foundation for future analytics and AI optimization.

---

22. Security & Access Control

The platform will implement appropriate application-level security measures.

Included

- Secure authentication.
- Email verification.
- Password protection.
- Role-based access control.
- Admin authorization.
- API validation.
- User data access restrictions.
- Basic input validation.
- Secure production configuration.

Sensitive credentials and third-party API keys will not be hardcoded into the application.

---

23. Testing & Quality Assurance

Testing will be performed throughout development and before production launch.

Testing Areas

- Authentication testing.
- Profile testing.
- Ride creation testing.
- Matching engine testing.
- Group capacity testing.
- Join/leave scenarios.
- Ride lifecycle testing.
- Fare calculation testing.
- Chat testing.
- Push notification testing.
- Rating testing.
- Admin panel testing.
- API testing.
- Cross-platform mobile testing.
- Regression testing.
- Production deployment testing.

Edge Cases

Testing will specifically cover scenarios such as:

- Group reaches maximum capacity.
- No compatible rider is available.
- Multiple compatible groups exist.
- Rider leaves a group.
- Rider cancels a ride.
- Group becomes inactive.
- Ride is cancelled.
- Invalid travel information.
- Duplicate ride requests.
- Notification failures.
- Chat/network interruptions.

---

24. Production Deployment

The development team will prepare the application for production deployment.

Included

- Production backend configuration.
- Database configuration.
- Environment configuration.
- Application build configuration.
- Production API configuration.
- Push notification configuration.
- Basic deployment validation.
- Release build preparation.

---

25. App Store & Google Play Submission Support

The development team will provide support for:

Apple App Store

- iOS production build.
- App configuration.
- Store submission preparation.
- Submission support.
- Required technical information.
- Resubmission support for issues within the agreed scope.

Google Play Store

- Android production build.
- Play Store configuration.
- Submission preparation.
- Submission support.
- Required technical information.
- Resubmission support for issues within the agreed scope.

App Store and Google Play review and approval timelines are controlled by Apple and Google and are not included within the development team's control.

---

26. MVP Development Effort

The revised MVP development estimate is:

Phase	Estimated Hours
Platform Setup & Architecture	35 Hours
Authentication & User Management	30 Hours
Ride Management & Rule-Based Matching	70 Hours
Real-Time Chat & Notifications	40 Hours
Backend Development & APIs	40 Hours
Web Admin Panel	30 Hours
Testing, Deployment & Store Submission	25 Hours
Total MVP Effort	**270 Hours**

---

27. Phase-Based Delivery Plan

Phase 1 — Foundation

Scope

- Project setup.
- React Native application setup.
- Backend setup.
- Database architecture.
- Authentication.
- University email verification.
- User profiles.
- Initial admin configuration.

Estimated Effort

65 Hours

Phase 2 — Ride Coordination

Scope

- Ride creation.
- Ride discovery.

- Rule-based matching.
- Group creation.
- Group capacity.
- Join/leave functionality.
- Booker workflow.
- Ride lifecycle.
- Fare calculation.
- Ride history.

Estimated Effort

70 Hours

Phase 3 — Communication & Administration

Scope

- Real-time chat.
- Push notifications.
- Ratings.
- Reliability score.
- Travel calendar.
- Admin panel.
- User management.
- Ride management.
- Group monitoring.
- University management.
- Airport management.
- Destination management.
- Notification management.

Estimated Effort

70 Hours

Phase 4 — Testing & Launch

Scope

- Functional QA.
- Integration testing.
- Cross-platform testing.
- Edge-case testing.

- Bug fixing.
- Regression testing.
- Production deployment.
- Release builds.
- App Store submission.
- Google Play submission support.

Estimated Effort

65 Hours

---

28. Total MVP Estimate

270 Development Hours

The 270-hour estimate covers the complete MVP scope described in this document.

The estimate includes:

- Cross-platform mobile application.
- Backend APIs.
- Database.
- Authentication.
- Student profiles.
- Ride creation.
- Ride discovery.
- Rule-based matching engine.
- Ride group management.
- Booker workflow.
- Fare split calculation.
- Real-time group chat.
- Push notifications.
- Ratings and reliability.
- Ride history.
- Travel calendar.
- Web admin panel.
- Testing.
- Deployment.
- App Store submission support.
- Google Play submission support.

---

29. AI Matching — Future Phase

AI optimization is **not included in the 270-hour MVP**.

The MVP will use the rule-based matching engine as the primary matching mechanism.

The architecture will keep the matching component modular so that an AI optimization layer can be added later.

Future AI Layer

The future AI system may evaluate:

- Multiple eligible ride groups.
- Pickup convenience.
- Rider compatibility.
- Historical ride behavior.
- Rider preferences.
- Group optimization.
- Travel efficiency.

Estimated Future Effort

40–60 additional hours

This future phase would include:

- AI service integration.
- Recommendation/ranking engine.
- AI request/response processing.
- Validation.
- Confidence checks.
- Fallback mechanism.
- Monitoring.
- Logging.
- Testing.
- Optimization.

The rule-based matching engine will remain available as the fallback mechanism.

---

30. Features Excluded from MVP

The following features are specifically excluded from the 270-hour MVP.

Payments

- Stripe integration.
- Credit/debit card processing.
- Automated payment collection.
- Automated reimbursement.
- Payment gateway webhooks.
- Automated refunds.
- Platform payment fees.
- Payment settlement.

Transportation Integrations

- Uber API.
- Lyft API.
- Direct ride booking.
- Fare retrieval from Uber/Lyft.
- Ride status synchronization.

Maps & Location

- Live GPS tracking.
- Real-time vehicle tracking.
- Google Maps route optimization.
- Advanced route planning.
- Turn-by-turn navigation.

AI

- AI ride optimization.
- AI recommendation ranking.
- Machine learning models.
- Predictive ride recommendations.

Marketing & Monetization

- Referral program.
- Rewards program.
- Promotional campaigns.
- Premium memberships.
- Subscription plans.
- Marketing automation.

Advanced Analytics

- Advanced analytics dashboard.
- Business intelligence reports.
- Predictive analytics.
- Advanced reporting.

Other

- Multi-language support.
- Advanced personalization.
- External CRM integrations.
- Advanced payment history.
- Automated financial reconciliation.

These features may be estimated and implemented separately in future phases.

---

31. Future Product Expansion

After the MVP has been validated, the platform can be expanded with:

Phase 2 Enhancements

- AI ride recommendations.
- Stripe payment processing.
- Payment history.
- Automated reimbursement.
- Uber integration.
- Lyft integration.
- Google Maps integration.
- Route optimization.
- Advanced analytics.

Phase 3 Enhancements

- Referral and rewards.
- Premium membership.
- Promotional campaigns.
- Marketing automation.
- Multi-university expansion.
- Business intelligence.
- Advanced AI optimization.
- Personalized ride recommendations.

---

32. Third-Party Services

The client will provide and maintain the required third-party accounts and services.

Potential services include:

- Apple Developer Account.
- Google Play Console.
- Firebase.
- Cloud infrastructure.
- Email service provider.
- Push notification services.
- AI API provider for future AI functionality.
- Payment provider for future payment functionality.
- Uber Developer account for future Uber integration.

Third-party subscription, licensing, transaction, API usage, hosting, and infrastructure costs are **not included** in the development estimate.

The development estimate covers implementation and integration only.

---

33. Client Responsibilities

The client will be responsible for providing:

- Apple Developer Account.
- Google Play Console Account.
- Firebase Project access.
- Required cloud/service credentials.
- University information.
- University email domains.
- Airport information.
- Destination information.
- Travel calendar/event information.
- Branding assets.
- Logo.
- App icons.
- Brand colors.
- Privacy Policy.
- Terms & Conditions.
- Required legal/compliance content.
- Testing feedback.
- Timely approvals.
- Final product approval.

Any delay in receiving required credentials, information, assets, feedback, or approvals may affect the project timeline.

---

34. Assumptions

This estimate is based on the following assumptions:

- React Native will be used for cross-platform mobile development.
- Firebase or an equivalent agreed service will be used for authentication, notifications, and applicable backend services.
- The client will provide required third-party accounts and credentials.
- The MVP will use rule-based matching.
- No AI optimization is required for MVP launch.
- No direct payment processing is required for MVP.
- No Uber/Lyft integration is required for MVP.
- No live GPS tracking is required.
- No advanced map routing is required.
- Business rules will be finalized before implementation of the relevant functionality.
- Client feedback will be provided in a timely manner.
- App Store and Google Play accounts will be available before submission.
- The MVP will use the agreed designs and workflows.
- Changes outside the documented scope will be estimated separately.

---

35. Change Request & Additional Scope

The 270-hour estimate is based on the functionality defined in this Scope of Work.

Any feature, workflow, integration, design change, or technical requirement that is not included in this document will be treated as additional scope.

Examples include:

- New third-party integrations.
- New user roles.
- Additional payment functionality.
- New matching rules requiring substantial architectural changes.
- Advanced analytics.
- New administrative modules.
- Major UI/UX redesigns.
- Additional platforms.
- New business workflows.

Additional requirements will be reviewed and estimated separately before implementation.

---

36. Project Timeline

The estimated development duration for the MVP is approximately:

8–10 Weeks

The timeline is based on:

- 270 development hours.
- Availability of required client information and credentials.
- Timely client feedback.
- Timely approval of completed phases.
- No major scope changes during development.

Development and calendar duration are different. The 270 hours represent estimated development effort, while the 8–10 week period represents the expected project delivery window.

---

37. App Store & Google Play Review

The development team will support the application submission process.

However, final approval is controlled by:

- Apple App Store.
- Google Play Store.

Review duration, approval, rejection, and additional compliance requirements are controlled by the respective platforms.

If Apple or Google requests technical changes related to the submitted application, the development team will provide reasonable support and resubmission assistance within the agreed project scope.

Any new functionality requested by the stores that falls outside the agreed MVP scope may require additional estimation.

---

38. Final MVP Deliverables

At the completion of the MVP, the project will include:

Mobile Application

- iOS application.
- Android application.
- Student authentication.
- Student profile.
- Ride creation.
- Ride discovery.
- Rule-based matching.
- Ride groups.
- Booker workflow.
- Fare split.
- Real-time chat.
- Push notifications.
- Ratings.
- Reliability score.
- Ride history.
- Travel calendar.

Backend

- Production-ready backend APIs.
- Database.
- Authentication.
- Matching engine.
- Ride management.
- Group management.
- Chat services.
- Notification services.
- Rating system.
- Fare calculation.
- Event logging.

Admin Panel

- Dashboard.
- User management.
- Ride management.
- Group monitoring.
- University management.
- Airport management.
- Destination management.
- Travel event management.

- Push notification management.
- Platform settings.

Launch

- Production deployment.
- iOS release build.
- Android release build.
- App Store submission support.
- Google Play submission support.
- Final QA and launch validation.

---

39. Final Scope Summary

The MVP is designed to validate the core student ride-sharing concept without introducing unnecessary complexity from AI optimization, online payments, or external transportation integrations.

The MVP will focus on the complete core journey:

Student Registration → Profile → Create Trip → Rule-Based Matching → Ride Group → Booker Coordination → Fare Split → Group Chat → Ride Completion → Rating & Reliability

The platform architecture will remain modular and scalable so that future functionality can be introduced progressively.

MVP Development Effort

270 Hours

Estimated Delivery Timeline

8–10 Weeks

Future AI Optimization

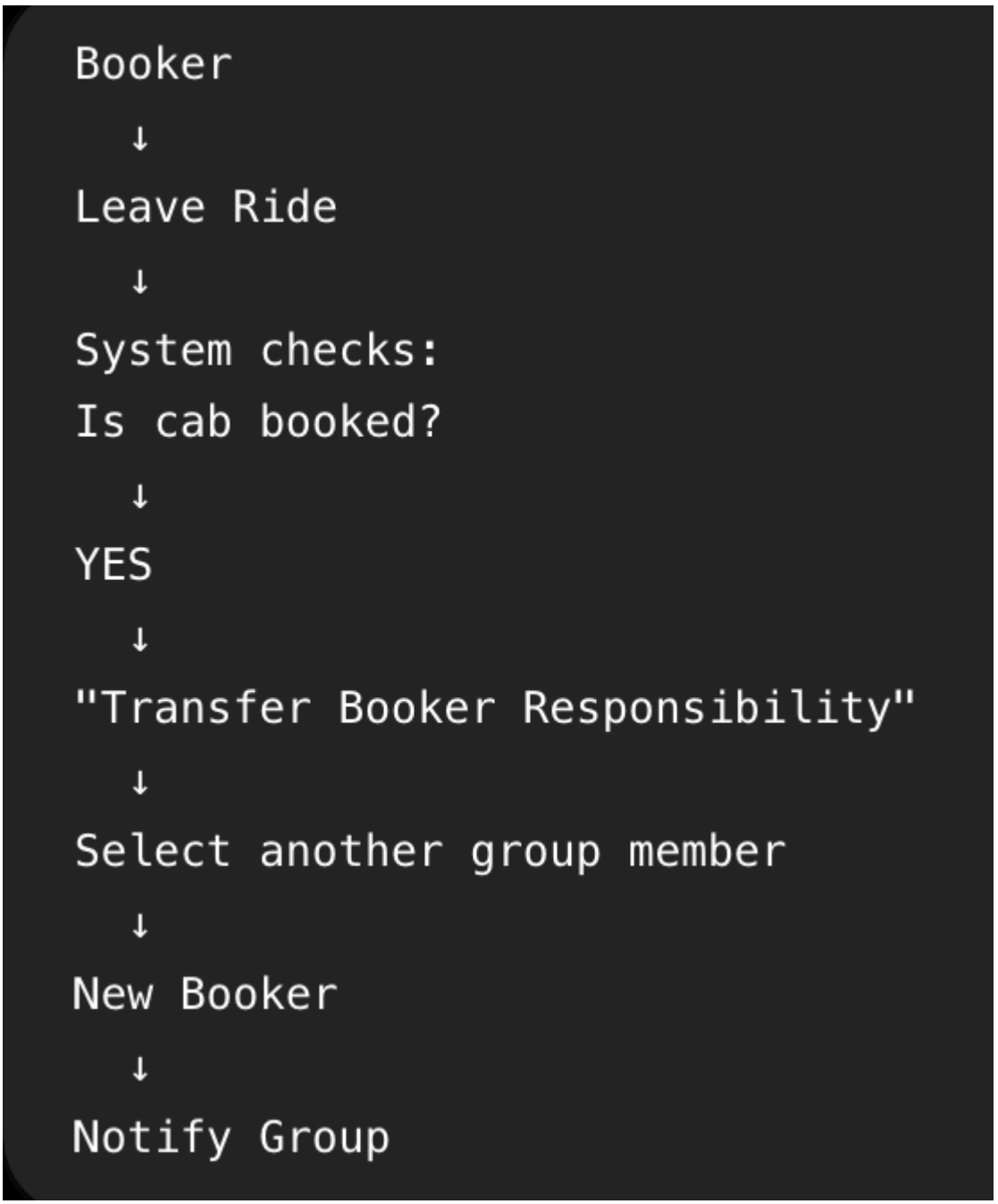
+40–60 Hours

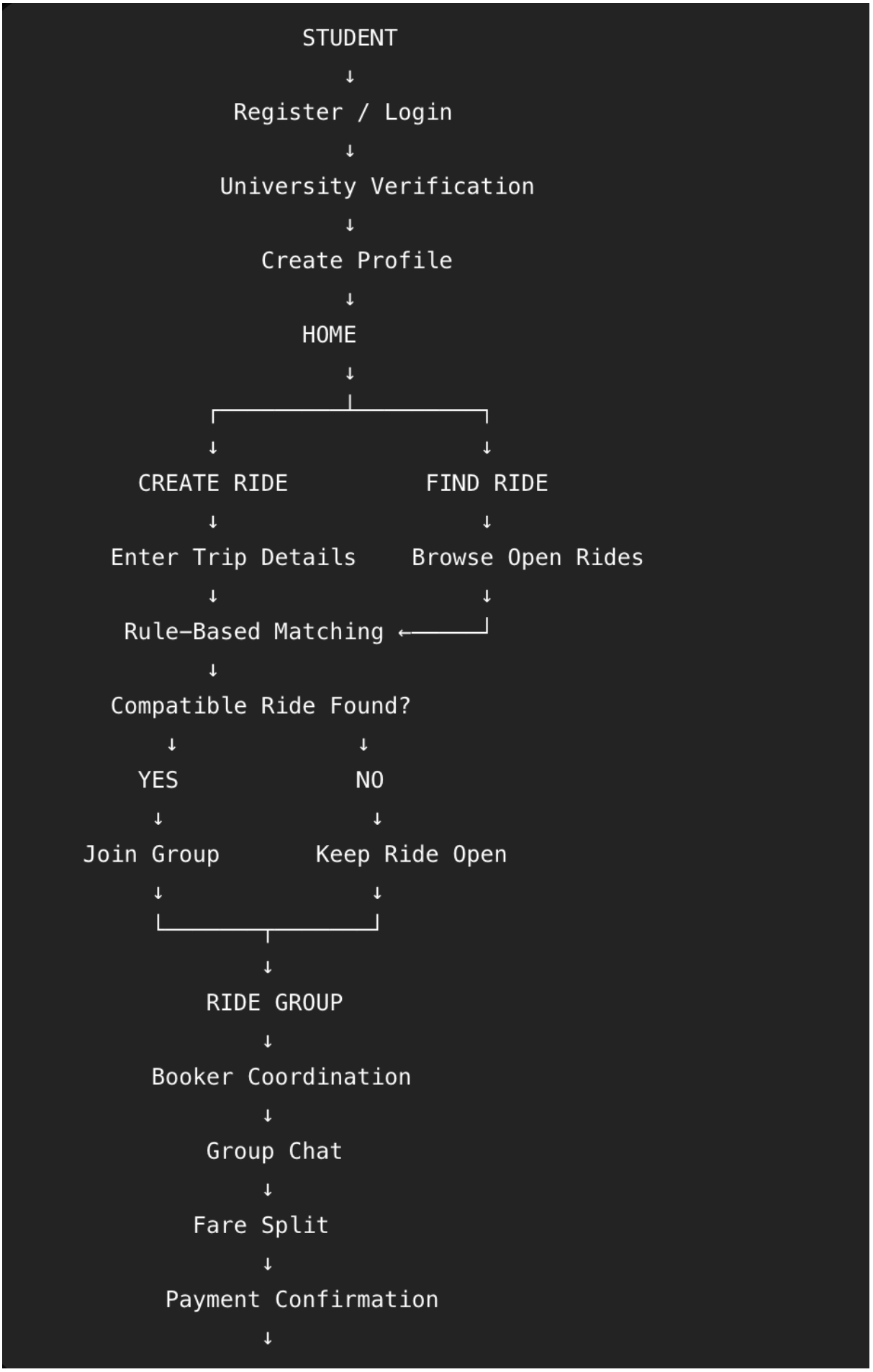
Future Full-Product Enhancements

+250–290 Hours

This phased approach allows the platform to launch with a focused MVP, validate student adoption and ride coordination behavior, collect useful ride history, and progressively introduce advanced capabilities such as AI optimization, online payments, Uber/Lyft integration, route optimization, and advanced analytics.

```
Ride Group Created
        ↓
Cab Fare Known
        ↓
Booker Enters/Confirms Fare
        ↓
System Calculates Individual Share
        ↓
Example:
Total Fare = ₹1,200
4 Riders = ₹300 each
        ↓
Each Rider Sees Their Share
        ↓
Rider Pays Booker Externally
        ↓
Rider Marks "Payment Confirmed"
        ↓
Booker/Group Can See Payment Status
        ↓
Ride Completed
```


Booker
↓
Leave Ride
↓
System checks:
Is cab booked?
↓
YES
↓
"Transfer Booker Responsibility"
↓
Select another group member
↓
New Booker
↓
Notify Group


STUDENT
Register / Login
University Verification
Create Profile
HOME
CREATE RIDE
FIND RIDE
Enter Trip Details
Browse Open Rides
Rule-Based Matching
Compatible Ride Found?
YES
NO
Join Group
Keep Ride Open
RIDE GROUP
Booker Coordination
Group Chat
Fare Split
Payment Confirmation

MVP — MAIN SIDE FLOWS

1. Rider Does Not Pay Booker

Ride group created → Booker books cab externally → Fare entered → Rider payment requested → Rider does not pay → Payment deadline reached → Payment marked **Overdue** → Booker can keep rider / request payment / remove rider → Group fare recalculated if rider is removed.

---

2. Rider Pays Late

Ride group created → Booker books cab → Rider payment pending → Deadline reached → Payment marked **Overdue** → Rider pays → Rider marks **Paid** → Booker confirms payment → Payment status becomes **Confirmed**.

---

3. Rider Leaves Before Cab Is Booked

Rider requests to leave → System checks cab booking status → Cab not booked → Rider confirms leaving → Rider removed from group → Group capacity updated → Fare recalculated → Group remains open for another rider.

---

4. Rider Leaves After Cab Is Booked

Rider requests to leave → System detects cab is already booked → Display **Late Cancellation Warning** → Rider confirms cancellation → Rider removed → Remaining riders notified → Fare recalculated → Remaining riders see updated fare share.

---

5. Rider Leaves After Paying Booker

Rider requests cancellation → Payment already confirmed → System records cancellation → Remaining group members notified → Fare recalculated → Refund/reimbursement handled manually outside the MVP.

---

6. Booker Leaves Before Cab Booking

Booker selects **Leave Ride** → System requires Booker transfer → Booker selects another group member → New Booker accepts → Booker responsibility transferred → Original Booker leaves.

---

7. Booker Leaves After Cab Booking

Booker selects **Leave Ride** → System detects cab is already booked → Booker must transfer responsibility → Another member accepts Booker role → Booking information transferred → New Booker becomes responsible for coordination → Original Booker leaves.

---

8. Nobody Accepts Booker Responsibility

Booker wants to leave → Transfer Booker requested → Other members reject/do not accept → System marks **Booker Replacement Required** → Admin notification → Admin assigns/requires a replacement Booker.

---

9. Booker Books Cab but Does Not Update App

Booker books cab externally → Does not enter booking information → System sends reminder → Booker updates booking details → Group notified.

If Booker still does not update → Group/Admin notified → **Booking Information Missing** status.

---

10. Booker Books Cab but Actual Fare Is Different

Estimated fare shown → Booker books cab externally → Actual fare entered → System compares estimated vs actual → Updated total fare → Individual shares recalculated → Group notified → Riders confirm updated amount.

---

11. Cab Gets Cancelled Externally

Booker books cab → External cab provider cancels → Booker marks booking cancelled → Group notified → Group returns to **Cab Booking Required** → Booker/new Booker arranges another cab → New fare entered → Group notified.

---

12. Cab Driver Is Delayed

Cab booked → Driver delayed → Booker updates group → Group receives notification → Members coordinate through chat → Ride remains active.

---

13. Flight Time Changes

Rider changes flight time → System re-checks matching compatibility → If still compatible → Group remains unchanged → If no longer compatible → Rider is warned → Group/matching status updated → Affected members notified.

---

14. Pickup Location Changes

Rider changes pickup location → System checks compatibility → If compatible → Update group → If incompatible → Warn rider → Re-matching or group change required.

---

15. New Rider Joins After Fare Calculation

Group already has fare calculated → New rider joins → Total rider count changes → Fare share recalculated → Existing members notified → New rider receives updated payment amount.

---

16. Rider Leaves and New Rider Joins

Rider leaves → Group capacity becomes available → Fare recalculated → New compatible rider found → New rider joins → Fare recalculated again → All members receive updated fare information.

---

17. No Compatible Rider Found

Student creates ride → Matching engine checks existing rides → No compatible group → Ride remains **Open** → Student receives notification when a compatible rider/group becomes available.

---

18. Multiple Compatible Groups Found

Student creates/finds ride → Multiple compatible groups identified → System ranks groups using configured rule-based criteria → User is shown eligible option(s) → User selects/join appropriate group → Group updated.

---

19. Group Becomes Full

Rider attempts to join → System checks capacity → Group is full → Joining is rejected → System searches for another compatible group → If none exists → User can create/keep their own ride open.

---

20. Entire Group Cancels

Group members cancel → Group no longer has enough riders → Group marked **Cancelled/Inactive** → Members notified → Ride removed from active rides → No further matching allowed.

---

21. Booker Does Not Book the Cab

Group confirmed → Booker receives booking responsibility → Booking deadline approaches → Reminder sent → Booker does not book → Group marked **Booking Required** → Members notified → Booker can continue or transfer responsibility → Admin intervention if necessary.

---

22. Payment Is Disputed

Rider marks payment as paid → Booker says payment was not received → Payment status becomes **Payment Disputed** → Booker and rider communicate through chat → Admin can review/resolve → Final status recorded.

---

23. Fare Is Disputed

Booker enters fare → Rider disputes amount → Rider can raise dispute → Booker provides clarification → Fare updated if required → All members notified → Admin intervention if unresolved.

---

24. Rider Is Removed Due to Non-Payment

Payment deadline reached → Rider has not paid → Payment marked **Overdue** → Booker selects **Remove Rider** → Rider removed → Remaining fare recalculated → Seat becomes available → Compatible replacement rider may join.

---

25. Ride Reaches Pickup Time

Ride reaches configured pickup time → Ride status changes to **In Progress** → Members receive notification → Chat remains available → Ride continues until marked completed.

---

26. Ride Is Completed

Ride completed → Booker/member marks ride completed → Ride status becomes **Completed** → Payment statuses finalized → Rating requests sent → Riders rate each other → Reliability scores updated → Ride moved to history.

---

27. Rider Does Not Submit Rating

Ride completed → Rating reminder sent → Rider does not rate → Rating remains unavailable/pending → Ride still moves to completed history.

---

28. Admin Intervention

Any unresolved situation such as:

- No Booker
- Payment dispute
- Fare dispute
- Abandoned group
- Booker unavailable
- User dispute

- Incorrect ride information

→ Admin reviews case → Admin can update/correct relevant ride/group information → Members notified where required.